

Maddy St. Charles
Climate 410, Lab 01
Due Monday, September 21st at 5pm

Methodology

Part 1: forestfire

In order to simulate the spread of wildfires in a given grid of area, I built two different functions that together model random fire activation spots and how the fire spreads from those spots, while also taking into account different levels of forestation. I decided to make two different functions because it allowed me to visualize the process more easily, while also giving me the ability to manipulate each variable explicitly when I put them in as parameters.

The first function I made is called forestfire. Its parameters are a 2D matrix of integers valued 1, 2, or 3 (representing the fire status of that particular forest cell), and prob_spread, a float that represents the probability that a fire will spread to the cells orthogonal to it (provided those cells have not been burnt already). The function works through a series of for loops that go through each cell in the given matrix, checking each one's value to determine if it's burnt, forested, or burning. It then proceeds to reassign the necessary values. If it finds a cell that is burning, it will check all the cells surrounding it orthogonally to see if they are forested or burnt. I accounted for edge cases by specifying index limits with logic statements as a condition for the if loop: before evaluating any cell's value, it first needs to make sure it's within the bounds of the matrix itself. If the cell meets the conditions and is forested, the function chooses a random float between 0 and 1 and decides either to change it to burning or to keep it as is, depending on the random number's value compared to the given prob_spread. If any of the cells are found to be burnt already, the fire will not spread to that cell and it will be left alone. In the end, the function returns the same sized matrix as the one input, but with replaced values corresponding to the fire. When called multiple times in a row, assigning a new variable to each iteration based on the previous run's returned matrix, forestfire shows a simulation of that particular forest's possible fire spread.

This function was tested by simply printing out the matrix it returned. Given a matrix fully forested (value 2) with specific dimensions of 3x3 and a single burning spot (value 3) in the center, I input it as the parameter and gave it a prob_spread of 1.0. With these conditions defined, after one iteration I expected to see a value of 1 in the center, 3s around the 1, and 2s everywhere else. A second iteration (and any beyond that) should change all the 3s to 1s, and the 2s surrounding the burning spots to 3s. I did get these results, so I proceeded to demonstrate it visually, as shown in Figures 1-3 below.

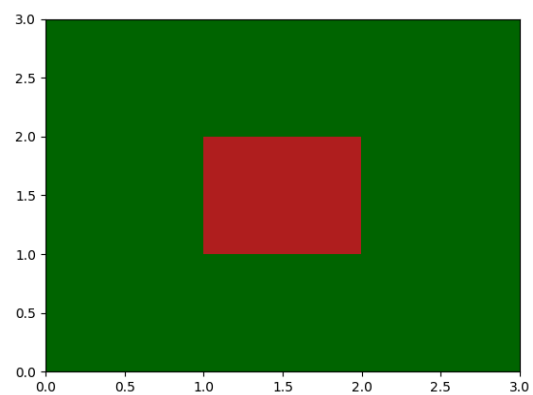


Figure 1: Initial grid

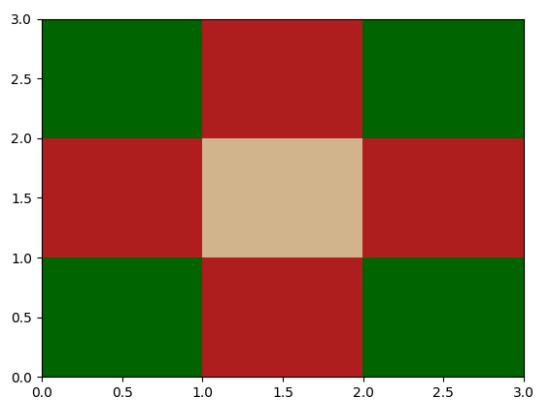


Figure 2: Iteration 1

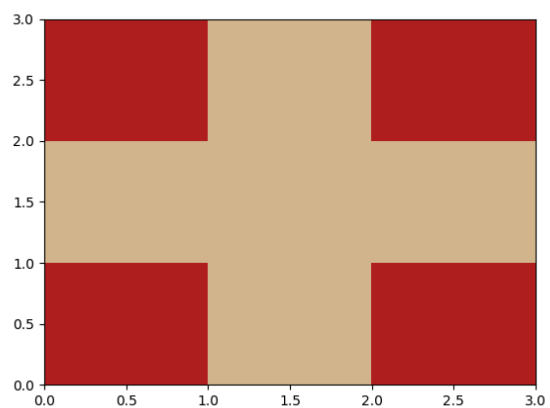


Figure 3: Iteration 2

Part 2: assign_random

The second function I wrote, `assign_random`, is actually meant to be used before `forestfire`, because it initializes the matrix that is put into `forestfire`. The parameters are a matrix of integers (the same as for `forestfire`), `prob_bare`, and `prob_ignite`. The value of `prob_bare` is a float that represents the probability of any random cell being burnt already, meaning the fire can't spread to it. `prob_ignite` is also a float, and it represents the probability of a forested cell catching fire on its own. When the matrix is put through `assign_random`, it will be returned with some of its cells being considered burnt or burning, so it can now be put into `forestfire` to simulate its spread based on these new conditions. The function randomly decides each cell's status by looping through each one in the matrix, choosing a random index, and assigning it either a 1 (burnt) or 3 (burning). The function returns the same sized matrix but with new values in some of the cells.

Part 3: assign_random_people and disease_spread

The two functions I wrote for the forest fires can also be applied to the spreading of disease, with just a couple variations in the code. The process is almost identical between the two applications. Mainly, I just renamed the variables to explicitly represent the elements of the disease context, but there was also an added step regarding the survival of a sick person. In the forest fire simulation, when a cell was actively experiencing the condition (on fire), the next iteration would leave it bare. Now, though, the person can either live through the disease or not, so I added a new probability called `prob_survive` that is put in as a parameter for `disease_spread`. This value affects the likelihood of a person surviving the disease and becoming immune. Other than this addition, the two disease functions are practically the same as their forest fire counterparts.

Part 4: Creating the figures

To create the figures, I followed the procedure given in the lab manual that demonstrated how to create gridded colormaps. I was then able to run matrices through each of my functions in order to plot them and visually investigate their results. I chose different colors for each map and condition. For the forest fire graphics, red indicates burning cells, green is forested, and tan is burnt. The disease spread graphics have brown for people who are sick, yellow for immune, blue for healthy, and gray for people who did not survive.

Analysis

Part 1: Wildfires

Varying `prob_ignite` and `prob_bare` impacts the fire's evolution because it determines the initial landscape the wildfire has to work with. Figure 4 shows one example result of putting a 3x5 forest grid into `assign_random`. Red shows a burning cell, tan is burnt, and green represents forested. I set both `prob_ignite` and `prob_bare` to

be 0.8, so there was an 80% chance for any of these cells to be ignited or bare initially. This matters because it determines how quickly a forest will get completely overtaken by a fire and left burnt: if the probability of ignition is higher and the probability of having a bare cell is lower, a forest will catch fire much faster than if the opposite was true. The results could be quantified based on the initial percentage of available forest. In this case below, 8 out of 15 cells (about 53%) are forested and available to burn.

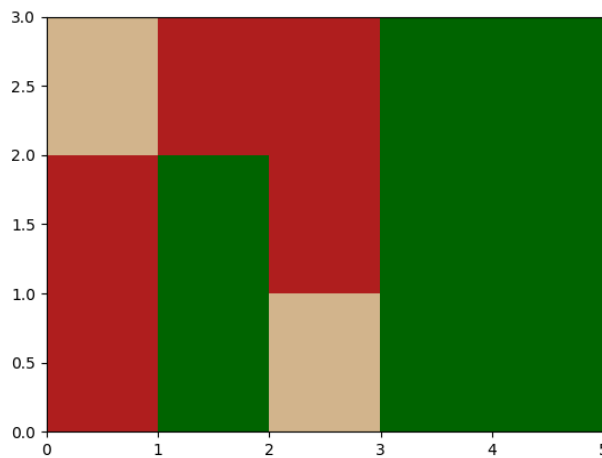


Figure 4

Controlled burns could potentially prevent wildfires from spreading, if the probability of spread was low. Igniting certain cells on purpose, knowing it's unlikely that they spread, could create a sort of protective ring around a forested area that would prevent outside fires from entering. For example, Figure 5 depicts a 5x5 forest with a ring around the center cell. If any fires start in the perimeter of the forest, the center cell is safe because the fire can't spread past the burnt part. However, if the probability of spreading is too high, it may not be helpful to ignite controlled burns. The fire could spread further than the intended region and potentially result in more damage than if a wildfire had just started on its own. So, the utility of a controlled burn seems to depend largely on the probability of spread: if it's unlikely that the fire will spread, it would prove useful, but a high likelihood of spreading would probably cause more harm than good.

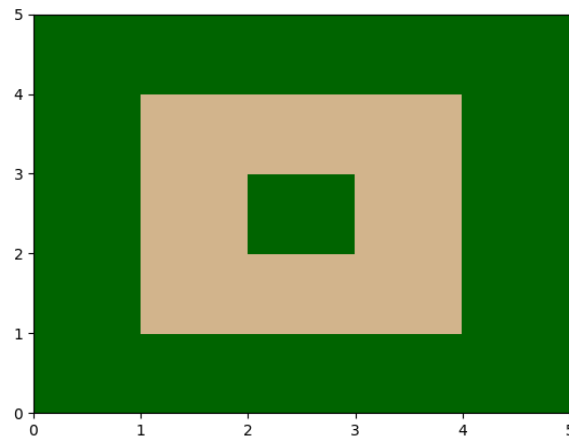


Figure 5

Part 2: Disease Spread

Mortality rate and early vaccine rates can have a notable impact on how a disease spreads throughout a population. Below two figures are shown: Figure 6 represents a population with no initial immunizations for a disease with a high mortality rate, and Figure 7 a population with more immunizations and a low mortality rate. Brown indicates a sick person, yellow an immune person, blue healthy/not yet affected, and gray a person who did not survive. Both scenarios have a 100% chance of spread and 50% chance of any person getting sick initially. On the left, I chose a probability of survival of 25% with 0% of people starting out immune. After a single iteration of my `disease_spread` function, only two people out of twentyfive remained healthy. In this specific situation, the disease proves to be very deadly and fast-acting. On the other hand, the scenario on the right had a 90% chance of survival with an 80% chance of someone being already immune. Through this iteration, many more people remained healthy. While there still are diseased people, there aren't as many, and they aren't going to cause as much population damage because there are immune people forming a similar kind of protective barrier as the controlled fires did in the forest fire analysis. Based on these results from the model, it could be concluded that both lower mortality rates and a higher proportion of vaccinated people are factors that can limit the spread and overall population resilience of a disease. With more people vaccinated and immune, it not only prevents themselves from getting the disease, but it can also prevent others from contracting it as well. Additionally, a lower mortality rate will definitely help more people survive a disease, because their likelihood of beating it is increased. Then, not only are they still alive, but they're now immune and not spreading the disease to those unaffected. Of course, this model is an oversimplification of how disease spreading actually works, but it is still a good way to visualize how a vaccination and low mortality rate can help a population against a disease.

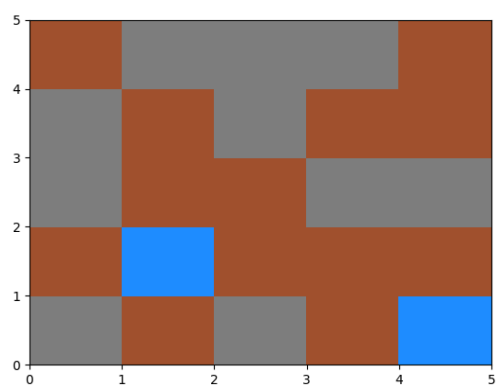


Figure 6

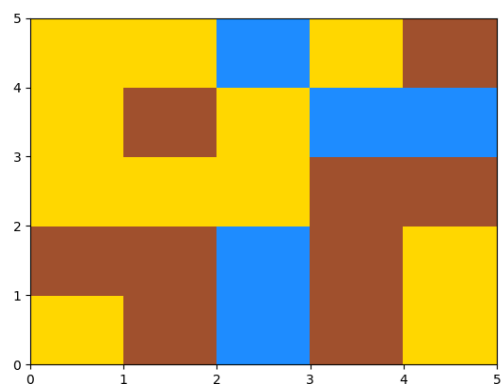


Figure 7